

This lab helped me to understand the different types of machine learning and how they are used for different purposes. The three main types I learned about were supervised, unsupervised, and reinforcement learning. Supervised learning uses labeled data, which means we give the model examples with known answers so it can predict things good. Unsupervised learning is different because the model looks for patterns or groups in data without being given the correct answers. Reinforcement learning involves an agent learning through trial and error by receiving rewards or penalties. Understanding these differences helped me see that machine learning is not one single process and that the type of learning depends on the problem being solved.

One of the most useful parts of the lab was working with the Wine dataset. It had information about all the different types of wine, and we used chemical features like alcohol, malic acid, ash, and alkalinity of ash to predict the wine's class. Before looking through training, I learned why it's important to explore the data and check for things such as missing values and relationships between features. Splitting the data into training and testing sets was also important because it allowed the models to be tested on data they had not already seen. As a result I understood the difference between a model actually learning patterns and simply memorizing the training data.

I also learned how different machine learning models can perform differently on the same dataset. In this lab, I worked with Logistic Regression and a Decision Tree. Logistic Regression performed better on the selected features, reaching an accuracy of 88.9%, while the Decision Tree performed not as good.

I think this means that choosing a model is not just about using the most complicated algorithm. The features being used and how well they relate to the target can have a major effect on the results. I also learned how a confusion matrix and classification report can be used to look at model performance in more detail instead of relying only on accuracy.

Overall, this lab gave me a little bit of a better understanding of the complete machine learning workflow in all its glory, from preparing and exploring data to training and evaluating a model.

Reflection Journal

Course / Lab: ITAI 1371 — Module 3: Machine Learning Workflow and Types of Learning

Group: ML_1371_14274 4

Student: Shareefah Adeola

Introduction

This lab helped me move from learning individual machine learning concepts to understanding how the different steps work together as a complete workflow. At first, the machine learning process seemed like a number of separate technical steps. As I worked through the notebook, I began to see how defining the problem, examining the data, selecting features, training a model, evaluating performance, and reflecting on the results are connected.

Understanding the Types of Learning

One of the most important concepts I learned was the difference between supervised, unsupervised, and reinforcement learning. Supervised learning uses labeled data to learn how to make predictions or classifications. Unsupervised learning looks for patterns or groups in data without labeled outcomes. Reinforcement learning is based on an agent interacting with an environment and learning from rewards or penalties. The distinction became clearer when I thought about whether the training data provides an answer or target for the model.

What I Learned from the Wine Classification Project

The wine classification project gave me an opportunity to see supervised learning in practice. The dataset contained 178 observations, 13 features, and three wine classes, with no missing values. I examined the data and visualizations before comparing Logistic Regression and Decision Tree models. Logistic Regression performed better in this lab, with an accuracy of 88.9%, compared with 83.3% for the Decision Tree. This showed me that different algorithms can perform differently on the same dataset.

Feature Selection and Model Performance

I also learned that the features included in a model can affect its performance. I experimented with alcohol, color intensity, and proline as selected features. The resulting model accuracy was 83.3%, lower than the original 88.9% accuracy. This helped me understand that reducing the number of features does not automatically improve a model. Some of the other available features may contain useful information for distinguishing between the wine classes.

Challenges and Problem-Solving

The most challenging part of the lab was understanding how to choose features and compare model performance. I also had to become more comfortable interpreting accuracy rather than simply running the code. Working through the notebook step by step helped me overcome this challenge. I learned to look at the code, understand what the model was doing, and use the results to guide the next step.

Connection to Banking and Finance

I can connect these concepts to banking and finance. A practical machine learning problem would be detecting fraudulent financial transactions and identifying unusual patterns in customer transactions. This would be a supervised classification problem when historical transactions are labeled as fraudulent or legitimate. Useful data

could include transaction amounts, dates and times, locations, payment methods, and information about previous fraudulent transactions.

Questions for Further Investigation

Two questions I would investigate further are: (1) How would using all 13 wine features affect the accuracy of the Logistic Regression model compared with using only three selected features? (2) How would other machine learning algorithms, such as Random Forest or Support Vector Machine, perform compared with Logistic Regression on the wine dataset? These questions would help me understand how feature selection and algorithm choice influence model performance.

Conclusion

Overall, this lab changed my view of machine learning from a collection of coding techniques into a structured problem-solving process. I learned that understanding the data and the problem is just as important as selecting an algorithm. I also learned that model performance should be evaluated and compared rather than assumed. Most importantly, I now have a clearer understanding of how supervised, unsupervised, and reinforcement learning differ and how machine learning can be connected to real-world applications in banking and finance.

Experimental Reflections

Before this experiment, I thought machine learning was mainly about using prebuilt libraries to process data. However, this experiment helped me understand that the complete six step workflow is a process of testing, making mistakes, and finding answers. When working with the Wine dataset, I learned more about the "garbage in, garbage out" principle. Feature selection is not random. For example, when we tried different feature combinations, such as using the first four dimensions or choosing different chemical components, the model accuracy changed a lot. This helped me understand that feature engineering is not only a technical task. It also requires us to understand the data and relationships between different features.

When I compared logistic regression and decision trees, logistic regression had a higher accuracy of 88.9%. At first, I was surprised because I thought the decision tree was a more complex model and should perform better. After looking at the misclassifications in the confusion matrix, I understood the reason better. When there are only a few features and the decision boundaries are more linear, simple models such as logistic regression can work better. They can also be less likely to overfit. This changed my previous idea that a more complex model is always better.

This experiment helped me understand how a model performs when it works with unseen data. In the future, I want to use these workflows with more complex and noisy real-world data. I also want to learn more about cross validation and hyperparameter tuning. These methods can help reduce overfitting and improve model performance.

A Reflection of the Wine Assignment in Lab 3

What looked at first like a pile of unrelated ceremony — scale this, split that, wrap the whole thing in a pipeline — turned out to be a sequence where every stage exists because the stage before it created a specific problem. Once I could name the problem, the step stopped feeling arbitrary.

I want to trace that sequence, because the order is the insight.

The assignment presents itself simply: pick three features from the thirteen available, train a logistic regression, compare the accuracy to a baseline. Three names in a list. That framing is a reasonable place to start, and I think it is deliberately where the notebook wants you to start, because it lets the real complexity surface one layer at a time rather than all at once.

The first layer surfaced immediately, in the form of a `KeyError`.

`df['alcohol', 'color_intensity', 'proline']` looks correct and is not. Python collapses those comma-separated names into a tuple, and pandas reads a tuple as one compound column label, then goes looking for a column that does not exist. The fix is to pass a list — or better, to pass `your_features`, the variable already defined two lines above for exactly this purpose.

This is a small thing, but it established the pattern for everything that followed. The gap between what I intended and what I wrote was invisible in the source and only became visible when something downstream refused to cooperate. Reusing `your_features` instead of retyping the names closes that gap permanently: now there is exactly one place where the feature choice lives, and the model and the printout cannot disagree about it.

That is the first step earning its place. Not style — a single source of truth.

With the indexing fixed, the code runs. It also, almost certainly, does not converge.

Alcohol content sits around 13. Proline runs well into the hundreds. Logistic regression's solver is descending a loss surface stretched grotesquely along one axis, and it exhausts `max_iter` before reaching a good solution. Critically, it does not fail. It fits, it predicts, and it prints an accuracy formatted to three decimal places that looks exactly as trustworthy as a correct one.

`StandardScaler` puts every feature on comparable footing, and the optimizer converges. This is where I started to see the logic of the workflow rather than just its steps: scaling is not a cosmetic preprocessing habit, it is what makes the next step — fitting — actually work. Skip it and the model you evaluate is not the model you think you trained.

And this step genuinely does improve performance, in the most literal sense. The accuracy goes up because the model is finally allowed to find its optimum.

Here is the part I found most satisfying, because it is a problem created by the previous solution.

The natural way to scale is to transform the whole feature matrix and then split it.

But `StandardScaler` computes a mean and standard deviation, and if it computes them across all 178 samples, those statistics have absorbed the test set. The test rows are supposed to represent data the model has never seen. Once their distribution has shaped the transformation applied to training data, they

no longer represent that.

`make_pipeline(StandardScaler(), LogisticRegression(...))` resolves it structurally. The pipeline fits the scaler on training data only and reuses those same statistics at predict time. I do not have to remember to do it right; the object does it.

The leak here is small and probably would not change my conclusions. But the same mistake with imputation, target encoding, or feature selection inflates results dramatically, and I would rather learn the habit on a problem where the stakes are low. Each step building on the last means the errors compound too — pipelines are how you stop that.

The final step is the one that reframed everything before it.

The notebook compares my accuracy against results['Logistic Regression']. For that comparison to say anything about feature choice, both models need identical preprocessing. If mine is scaled and the baseline is not, the difference measures scaling, not features — I would be answering the wrong question with real confidence.

And the test set is 36 samples. One flipped prediction moves accuracy by 2.8 points. A four-point gap between two feature sets is well inside the range that a different `random_state` could produce on its own. Three decimal places imply a resolution the experiment cannot deliver.

`cross_val_score(model, X, y, cv=5)` gives every sample a turn as test data and returns a spread rather than a single point. Looking at the mean and the standard deviation together, I can finally say whether a difference is real.

This is the step that makes all the others worthwhile. Correct indexing, converged fitting, and leak-free preprocessing produce a number — evaluation is what tells me whether the number deserves to be believed.

Looking back, none of these steps are optional and none of them are interchangeable. You cannot evaluate a model that did not converge. You cannot trust convergence achieved through leakage. You cannot detect leakage if your metric is too noisy to resolve the difference. They stack.

The modeling call itself — one line, `LogisticRegression()` — is the part I understood from the start and the part that turned out to matter least. Everything that makes a result trustworthy happens around it, and I now think of that surrounding work as the actual discipline rather than overhead attached to it.

I still do not know which three features are best. But I know how I would find out, and more importantly I know how I would tell whether my answer was real. A week ago I would have read the first accuracy the notebook printed and believed it.